

# Take-Home Assessment: Multi-Agent Agentic Framework (Open Source)

## Objective

Design and implement an **API-driven open-source multi-agent agentic framework** capable of solving a wide range of user queries by intelligently planning tasks, selecting the right execution path, and calling MCP tools when required.

This assessment is intended to evaluate your **system design thinking, agent architecture decisions, reasoning ability, planning approach, and implementation quality**—not just code completion.

We are intentionally testing how you think through real-world production challenges such as:

- tool orchestration
- task decomposition
- bulk document operations
- context limitations
- structured planning
- streaming execution visibility
- agent delegation
- scalable execution patterns

Perfect completion is **not expected**. Strong architecture and thoughtful tradeoff decisions matter more than feature completeness.

---

## Problem Context

You are provided with a **fake enterprise document dataset** containing **9,000 rows**, where each row represents a document and its metadata.

Each document contains:

- `content_id`
- `container_id`
- `document_name`
- document metadata
- AI-generated insights:
  - Document Classification
  - Summary

- Named Entity Recognition (NER)
- Keywords

This simulates a real enterprise document intelligence platform.

---

## Sample Dataset

Example:

```
df.head(5)
```

Each row includes fields such as:

- document metadata
  - document classification
  - summaries
  - extracted PII (NER)
  - keywords
  - timestamps
  - language
  - file type
  - status
- 

## Provided MCP Tools (FastMCP Preferred)

You are required to create and integrate the following **4 MCP tools** as deployable MCP Server tools:

### Required Tools

#### 1. `get_document_insights`

Retrieve AI-generated insights for documents:

- classification
- summarization
- NER / PII
- keywords

Supports:

- single document mode
  - all documents in container mode
- 

## **2. `get_active_documents_metadata`**

Retrieve all active documents inside a container.

Supports filtering and enables bulk workflows such as:

- translating all legal documents
  - processing all financial documents
  - filtering by language/category/status
- 

## **3. `translate_document_preserving_structure`**

Translate one or many documents while preserving:

- formatting
- layout
- structure

Supports:

- single document translation
- bulk translation mode

Bulk translation should handle:

- concurrency control
  - batching
  - failure handling
  - partial success reporting
- 

## **4. `aiagent`**

Document RAG-based Q&A tool.

Used for:

- answering questions from document corpus
- summaries
- clause extraction
- risk analysis
- comparison
- general document intelligence

This should be treated as the default document Q&A tool.

---

## Core Task

Build an **OpenCode-style API-driven Agentic System (NOT a TUI application)** that accepts user input and autonomously solves the request.

This system should behave like a production-grade agent framework capable of:

- task planning
  - tool orchestration
  - multi-step execution
  - long-running task handling
  - clarification loops when required
  - bulk operations across massive datasets
  - code execution where appropriate
  - offloading large operations instead of keeping everything in LLM memory
- 

## Requirements

### 1. API-Driven Architecture (Mandatory)

This must be an **API-driven agentic framework**, similar to:

- OpenCode
- DeepAgents
- Hermes Agent

### Explicitly NOT Allowed

- Terminal UI (TUI)
- CLI-only workflows
- interactive shell-only agents

The system should expose programmatic execution flows.

---

## 2. Use Hermes Agent / OpenCode Style Execution (Mandatory)

Use concepts inspired by:

- Hermes Agent
- OpenCode
- DeepAgents

Examples:

- execution engine
- worker agents
- code executor
- tool executor
- spawning parallel sub agents
- support bulk operations (calling translate tool on 9000 documents)

Do **not** build a simple FastAPI wrapper around tools.

We are looking for true agentic orchestration.

---

## 3. Task Planning Before Execution (Plus)

Before executing tasks, your system must create a plan.

Refer to:

- DeepAgents TODO planning
- execution DAGs
- dependency graphs
- task decomposition

Example:

User asks:

Translate all my financial documents to German

Expected planning:

1. Identify target container
2. Retrieve active documents
3. Filter financial documents
4. Validate document count
5. Execute bulk translation
6. Track progress
7. Return summary

Planning should be visible and structured.

---

## 4. Plan Mode (Interrogation Mode) (Plus)

Your system must support a **Plan Mode** where the agent asks clarifying questions before execution if the request is ambiguous.

Example:

User says:

Create a dashboard from my documents

Agent should ask:

- HTML dashboard or React dashboard?
- Should it be downloadable?
- Which document categories should be included?
- Do you want charts only or insights + charts?

The system should not blindly execute unclear requests.

---

## 5. Streaming Execution Updates (Plus)

Use: **Server-Sent Events (SSE)**

for real-time execution streaming.

We want to see updates such as:

Planning started...

```
Tool selected: get_active_documents_metadata
Retrieved 12,450 documents
Filtering category=financial
Executing bulk translation...
Progress: 1,000 / 12,450
Completed successfully
```

This is extremely important.

---

## 6. Large-Scale Execution Strategy (Mandatory)

The framework must handle:

### Small Tasks

Single document operations

### Massive Tasks

Bulk operations across:

**1,000,000+ documents**

Since agents cannot keep such scale in memory, your system should intelligently:

- offload to code execution
- use batching
- use worker execution patterns
- avoid context explosion
- avoid naive tool loops inside the LLM

This is a major part of the assessment.

---

## Example User Queries

Your system should be able to handle queries such as:

- Can you translate all my financial documents in my container?
- What are my payment terms?
- Can you create an HTML dashboard from all my documents?
- What is the latest news on AI?

- Can you convert all my PDF documents to DOCX?
  - Find all documents containing high-risk indemnification clauses
  - Generate a summary report of all legal agreements with high PII exposure
- 

# Expected Deliverables

Please submit:

---

## 1. Architecture Design

Include:

- agent architecture
- planning engine
- task decomposition logic
- execution engine
- MCP orchestration strategy
- state management
- long-running task handling
- bulk execution strategy
- failure recovery
- retries
- observability

Diagrams are highly encouraged.

---

## 2. Working Code

A runnable prototype is expected.

It does not need to be fully production-ready, but should demonstrate:

- planning
  - agent routing
  - MCP tool execution
  - multi-step workflows
-



## 3. Design Decisions

Explain:

- why you chose this architecture
  - tradeoffs considered
  - why you avoided alternative approaches
  - scaling considerations
  - production concerns
- 

## Evaluation Criteria

We will primarily evaluate:

### System Thinking

- Can you design agent systems for real-world scale?

### Agent Architecture

- Do you understand true agentic execution beyond prompt wrappers?

### Planning Quality

- Can your agent think before acting?

### Scalability

- Can your system handle million-document operations intelligently?

### MCP Integration

- Can you use tools correctly and efficiently?

### Streaming UX

- Can users observe execution in real time?

## Bonus Points

Strong bonus if you include:

- Use an API-driven agentic framework inspired by Hermes Agent / OpenCode, not a FastAPI-based TUI or simple request-response wrapper. The system should support true agent orchestration with planning, execution, tool usage, and multi-step task handling.
- Provide real-time execution visibility using Server-Sent Events (SSE), allowing users to track planning, tool selection, task progress, execution status, and final completion updates as the workflow runs.
- Implement structured Task Planning before execution, similar to DeepAgents TODO planning. The agent should first decompose the user request into clear executable steps, identify dependencies, select the appropriate tools, and define the optimal execution path before starting any action.
- Support a dedicated Plan Mode (interrogation mode) where the agent asks clarifying questions. Instead of making assumptions, the system should gather missing details required for accurate execution before proceeding.

---

## Final Note

This is intentionally designed to be challenging.

We do not expect perfection.

We want to understand:

- *How you think.*
- *How you design.*
- *How you solve ambiguity.*
- *How you handle scale.*
- *How you approach production agent systems.*